12/9 Componentes de Proyecto SW y Estimaciones de Sw (debrief)

05 Componentes de Proyecto SW

Un proyecto de software, al igual que cualquier esfuerzo de ingeniería, es una empresa compleja que se compone de múltiples elementos interrelacionados. Para asegurar el éxito, estos elementos deben gestionarse de manera efectiva, desde la concepción hasta el despliegue y mantenimiento.

Resumen Ejecutivo

La Ingeniería de Software profesional se fundamenta en la gestión eficaz de proyectos, los cuales son esfuerzos temporales y únicos destinados a crear un producto o servicio. Estos apuntes han desglosado los **componentes clave de un proyecto de software**, estructurándolos en las "Cuatro P" de Pressman: **Personal, Producto, Proceso y Proyecto**.

El **Personal** es el activo más valioso; el éxito del proyecto depende de un equipo cohesionado, motivado y con habilidades complementarias, guiado por un liderazgo adaptable. La capacitación y la comunicación abierta son prácticas esenciales, mientras que la falta de atención a las dinámicas del equipo es un error común. Los enfoques ágiles, como XP, enfatizan la autoorganización y la colaboración constante.

El **Producto** se refiere a lo que se construirá, definido por el ámbito y los requerimientos. La ingeniería de requerimientos es crítica, diferenciando entre requisitos funcionales (qué hace el sistema) y no funcionales (cómo se desempeña). La especificación detallada en documentos (SRS) es vital para sistemas grandes, aunque los métodos ágiles prefieran historias de usuario más flexibles. La administración de requerimientos y el aseguramiento de la calidad (ACS) son actividades continuas para mantener la integridad y fiabilidad del software.

El **Proceso** describe el "cómo" se construye el software. Desde modelos prescriptivos como la Cascada (secuencial y documentada, ideal para requisitos estables) hasta los métodos ágiles como Scrum y XP (iterativos, adaptativos, con énfasis en el software funcionando y la respuesta al cambio), la elección del proceso debe ser flexible y contextual. Actividades marco como la comunicación, planificación, modelado, construcción y despliegue, junto con actividades sombrilla como las revisiones técnicas y la gestión de riesgos, son universales. La mejora continua de procesos es clave para optimizar la calidad.

Finalmente, la gestión del **Proyecto** abarca la planificación, la administración de riesgos y la medición. Una planificación rigurosa es el "mapa" del proyecto, mientras que la administración de riesgos identifica, proyecta y mitiga posibles problemas. Las métricas de software, incluidas las líneas de referencia, son esenciales para medir el progreso, la productividad y la calidad, permitiendo ajustes informados.

Respecto a estimaciones de SW:

- **Estimación ≠ planificación ≠ compromiso.** Se usan para informar el plan, no para prometer fechas duras.
- **Cinco familias de métodos:**
  1. **Experiencia:**
     - **Históricos:** proyectás con productividad de proyectos comparables.
     - **Juicio experto:** WBS + **PERT = (o+4·m+p)/6**.
     - **Wideband-Delphi:** varias rondas anónimas hasta converger.

    - **Analogía:** ajustás contra 1–3 proyectos realmente similares.
2. **Recursos (capacidad):** fechas/esfuerzo según plantel y ventana disponibles; útil con presupuesto fijo, **riesgo** de ignorar alcance.
3. **Mercado (capacidad del cliente):** precio/tope manda; **riesgo** de subestimar y degradar calidad/scope.
4. **Componentes / Proceso:** descomponer (top-down/bottom-up) y/o repartir por actividades (req/diseño/código/pruebas) y sumar.
5. **Algorítmicos:** fórmulas con tamaño + multiplicadores (equipo, complejidad); **siempre calibrar** con datos propios.

En conjunto, la comprensión de los componentes del proyecto y las técnicas de estimación proporciona a los ingenieros y gerentes las herramientas necesarias para enfrentar la complejidad del desarrollo de software, optimizar recursos, gestionar riesgos y, en última instancia, entregar soluciones de alta calidad que satisfagan las necesidades del cliente.

---

1. Definición y Características de un Proyecto de Software

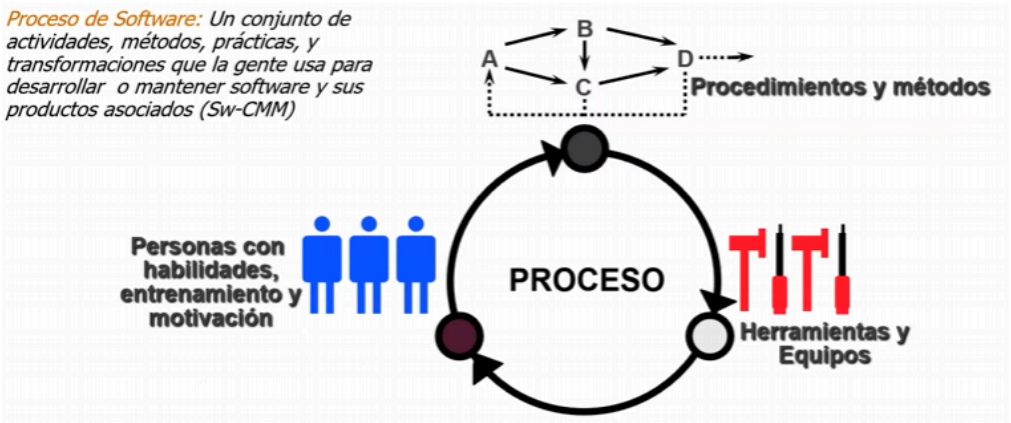
[Síntesis] Un proyecto de software es una iniciativa con un principio y un fin definidos, orientada a la creación de un producto o servicio singular. Su éxito depende de cumplir con el tiempo y presupuesto establecidos, además de satisfacer los requerimientos del cliente. No se trata solo del código, sino de todo el "conocimiento empaquetado" y la documentación asociada.

[Definición] **Un proyecto de software es un esfuerzo temporal que se lleva a cabo para crear un producto, servicio o resultado único.** Más académicamente, es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para "tener el trabajo hecho" a tiempo, con el presupuesto acordado y habiendo satisfecho las especificaciones o requerimientos. [Referencia].

- **Características Clave:**
    - **Unicidad**: Cada proyecto, por similar que sea a otros, posee características que lo hacen singular, como cronogramas diferentes.
    - **Orientado a objetivos**: Los proyectos deben tener objetivos claros y no ambiguos que guíen todo el esfuerzo.
    - **Temporalidad**: Todo proyecto tiene un inicio y un fin definidos, lo que lo diferencia de las operaciones continuas.
    - **Complejidad**: Los proyectos de software son inherentemente complicados y demandan una gestión estructurada.
    - **Producto Completo**: Un producto de software profesionalmente desarrollado va más allá de los programas de computadora; incluye también archivos de configuración, documentación del sistema (describiendo su estructura), documentación del usuario (explicando cómo usarlo) y, a menudo, sitios web con información actualizada. Es decir, el software es "conocimiento empaquetado a distintos niveles de abstracción".

2. Elementos Clave de un Proyecto de Software: Las "Cuatro P"

[Síntesis] Roger Pressman identifica cuatro elementos esenciales para la gestión de proyectos de software, conocidos como las "Cuatro P": **Personal, Producto, Proceso y Proyecto.** Este orden es deliberado, priorizando a las personas, seguido de lo que se construye, cómo se construye y, finalmente, la gestión del esfuerzo global. Un error en cualquiera de estas áreas puede ser una causa de fracaso.



2.1. El Personal (People)

[Síntesis] El capital humano es el **factor más crítico** en la ingeniería de software, una actividad intensiva en personas. **El éxito del proyecto depende de la selección de un equipo** con habilidades complementarias, una comunicación efectiva y una motivación constante. Un líder de proyecto debe ser adaptable y entender la organización para guiar al equipo. [Referencia].

- **2.1.1. Roles y responsabilidades del equipo**:
  - [Definición] Un grupo de proyecto necesita una *combinación de personas con diversas habilidades* (ej. negociación con clientes, programación, pruebas, documentación).
  - [Buena práctica] Para proyectos grandes, se construyen tablas que explicitan los roles y quiénes los desempeñan, aunque una persona puede asumir múltiples roles o un rol puede ser compartido.
  - [Conexiones] La selección del personal es crítica, ya que las capacidades de los integrantes impactan directamente en el éxito.
- **2.1.2. Motivación y dinámicas de equipo**:
  - [Buena práctica] Los ingenieros de software suelen estar motivados por el trabajo en sí. La **cohesión del equipo es vital,** y un grupo con **personalidades complementarias** (orientadas a la tarea y a la interacción) funciona mejor. Actividades como involucrar a todos en la especificación y el diseño, y organizar almuerzos informales o días de actualización tecnológica, fomentan la colaboración y fortalecen los lazos.
  - [Error común] Ignorar las personalidades y la comunicación puede llevar a problemas, como miembros que no cumplen con las expectativas o que priorizan desafíos técnicos sobre los estándares del proyecto.
  - [Buena práctica] La capacitación del personal es indispensable; "si cree que la capacitación es cara, pruebe con la ignorancia".
  - [Métrica] Se pueden establecer métricas para el personal, aunque no se detalla en las fuentes cómo, se sabe que existen modelos como el CMM de personal.
- **2.1.3. Enfoques Ágiles sobre el Personal**:
  - [Comparación] En el desarrollo ágil, se valora a los individuos y las interacciones sobre los procesos y las herramientas. Los equipos ágiles son pequeños, altamente motivados y autoorganizados, fomentando el aprendizaje continuo. La programación extrema (XP) enfatiza el trabajo en parejas, lo que mejora la calidad del código y la transferencia de conocimiento. [Referencia].

2.2. El Producto (Product)

[Síntesis] El **producto de software define lo que se construirá, abarcando su funcionalidad, características, datos, rendimiento y restricciones.** La ingeniería de requerimientos es una fase crucial, ya que una definición deficiente puede generar errores costosos. La calidad del software es subjetiva y debe ser planificada y asegurada a lo largo de todo el proceso. [Referencia].

- **2.2.1. Ámbito del Software y Requerimientos**:
  - [Definición] El **ámbito del software describe el alcance del producto, incluyendo qué funcionalidades se entregarán, los datos de entrada/salida, el contenido, el rendimiento, las restricciones, las interfaces y la confiabilidad.** Un enunciado preliminar del ámbito debe expandirse para incluir detalles concretos y delimitación cuantitativa antes de comenzar la estimación.
  - [Conexiones] **Es la base para la estimación del esfuerzo.**
  - [Riesgo] **Un ámbito mal definido es una causa común de problemas en proyectos**.
- **2.2.2. Tipos de Requerimientos (Funcionales y No Funcionales)**:
  - [Definición] La **ingeniería de requerimientos** es el proceso de comprender y definir qué servicios se requieren del sistema y las restricciones sobre su operación y desarrollo. Es una etapa crítica, ya que los errores aquí pueden propagarse y causar problemas en etapas posteriores del PUD, RUP o Agil.
  - **Requerimientos funcionales**: Describen los servicios que el sistema debe proporcionar.
  - **Requerimientos no funcionales**: Especifican restricciones operacionales y de desarrollo, como fiabilidad, protección, rendimiento, usabilidad, seguridad y confidencialidad. Son cruciales para sistemas críticos.
- **2.2.3. Documento de Requerimientos de Software (SRS o ERS) y Especificación**:
  - [Definición] El **Documento de Requerimientos de Software (SRS o ERS) es una declaración formal (Cliente y developers) y oficial de lo que los desarrolladores deben implementar y entregar al cliente.** Contiene requerimientos de usuario (alto nivel, para clientes/usuarios) y requerimientos del sistema (detallados, para desarrolladores).
  - [Buena práctica] Una **estructura sugerida (IEEE)** incluye prefacio, introducción, requerimientos funcionales, no funcionales, interfaces externas, requerimientos de datos, etc..
  - [Buena práctica] La **especificación de requerimientos** debe ser clara, no ambigua, fácil de entender, completa y consistente (aunque difícil de lograr).
    - **Lenguaje Natural**: Expresivo e intuitivo, pero propenso a ambigüedades.
    - **Lenguaje Natural Estructurado**: Limita la ambigüedad mediante plantillas y formas estándar.
    - **Escenarios y Casos de Uso**: Ejemplos concretos de interacción del sistema. Los **casos de uso** identifican actores y el tipo de interacción. Son útiles para detallar descripciones y priorizar requerimientos, especialmente en métodos ágiles. Se documentan con diagramas UML.
  - [Comparación] **Los métodos ágiles argumentan que los requerimientos cambian tan rápidamente que un documento de requerimientos se vuelve obsoleto, prefiriendo historias de usuario en tarjetas.** Sin embargo, para sistemas grandes y críticos, un documento formal sigue siendo útil, especialmente para requerimientos de negocio y fiabilidad que se aplican al sistema en su conjunto.
- **2.2.4. Administración de Requerimientos**:
  - [Definición] Es el conjunto de actividades para identificar, controlar y dar seguimiento a los requerimientos y sus cambios. Es esencial debido a la naturaleza cambiante de los requisitos.
  - [Conexiones] Es similar a la administración de configuración de software. La trazabilidad es clave.

- **2.2.5. Calidad del Software y Aseguramiento (ACS)**:
  - [Definición] La **calidad del software** es subjetiva y depende de los atributos no funcionales (ej. fiabilidad, usabilidad, eficiencia). Un plan de calidad debe definir los atributos más importantes para un sistema específico.
  - [Control de Calidad] Acciones de ingeniería de software para asegurar que los productos del trabajo cumplen las metas de calidad, incluyendo la revisión de modelos y la inspección del código.
  - [Definición] El **Aseguramiento de la Calidad del Software (ACS)** establece la infraestructura para métodos sólidos de ingeniería de software, administración de proyectos y control de calidad. Incluye auditorías y reportes para evaluar la efectividad.
  - [Buena práctica] El equipo de ACS debe ser independiente del equipo de desarrollo para objetividad y para reportar la calidad sin estar influido por conflictos de desarrollo. Para compañías más pequeñas esto puede ser imposible.
  - [Métrica] La calidad se mide indirectamente a través de índices y métricas. Los atributos de calidad externos (observables por usuarios: velocidad, confiabilidad, usabilidad) y atributos internos (importantes para ingenieros: diseño) son clave. [Referencia].

2.3. El Proceso (Process)

[Síntesis] El proceso de software es un **conjunto adaptable de actividades, acciones y tareas para crear un producto, no una receta rígida. Define "quién hace qué, cuándo y cómo".** Existen **modelos prescriptivos** (tradicionales) que enfatizan el orden y la consistencia, y métodos **ágiles** que priorizan la adaptabilidad y la respuesta al cambio. [Referencia].

- **2.3.1. Actividades Marco General y Actividades Sombrilla**:
  - [Definición] **Actividades Marco General (Pressman)**:
    1. **Comunicación**: Interacción con el cliente para entender objetivos y reunir requerimientos.
    2. **Planificación**: Crear un "mapa" (plan del proyecto) que guía al equipo, define tareas, riesgos, recursos y programación.
    3. **Modelado**: Crear "bosquejos" o modelos para entender mejor los requerimientos y el diseño.
    4. **Construcción**: Incluye la generación de código y las pruebas iniciales.
    5. **Despliegue**: Entrega del software al usuario y obtención de retroalimentación. [Referencia].
  - [Definición IMP!!] **Actividades Sombrilla**: ***Son aplicables a lo largo de todo el proceso e incluyen: aseguramiento de la calidad del software, revisiones técnicas, medición, administración de configuración y administración del riesgo***. [Referencia].
- **2.3.2. Modelos de Proceso de Software (Cascada, Incremental, Espiral, RUP, Ágiles)**:
  - [Definición] **Modelo en Cascada (Tradicional)**: Sigue una secuencia ordenada de pasos (comunicación, planificación, modelado, construcción, despliegue), con revisiones al final de cada fase y gran dependencia de la documentación.
    - [Ventajas] Permite encontrar errores en etapas tempranas; útil cuando los requisitos son estables y la calidad es prioritaria; bueno para equipos con poca experiencia.
    - [Desventajas] Exceso de documentación, falta de resultados hasta el final, inflexibilidad ante cambios de requisitos.
    - [Ejemplo] Un sistema de control para una nave espacial, donde los requisitos de seguridad y fiabilidad son extremadamente altos y estables.

- [Definición] **Modelo Incremental**: Produce versiones cada vez más completas del software en iteraciones.
- [Definición] **Modelo en Espiral**: Reconoce explícitamente el riesgo. Cada ciclo implica definir objetivos, valorar alternativas, identificar y resolver riesgos, y planificar la siguiente fase. La minimización del riesgo es una actividad central. Es un modelo de proceso evolutivo.
- [Definición] ***Proceso Unificado Racional (RUP):*** Impulsado por casos de uso, centrado en la arquitectura, iterativo e incremental. Fases: Concepción (establecer caso empresarial), Elaboración (comprender dominio, arquitectura, plan, riesgos), Construcción y Transición. [Referencia].
- [Definición] ***Métodos Ágiles:*** Valores: individuos e interacciones sobre procesos y herramientas; software funcionando sobre documentación exhaustiva; colaboración con el cliente sobre negociación contractual; respuesta al cambio sobre seguir un plan.
  - **Programación Extrema (XP)**: Destaca el desarrollo dirigido por pruebas (escribir pruebas antes del código), pruebas incrementales, participación del usuario en pruebas de aceptación y automatización de pruebas. Prioriza código legible y de calidad sobre documentación formal exhaustiva.
  - **Scrum**: Se enfoca en la gestión de prioridades del proyecto, unidades de trabajo agrupadas (backlog), comunicación y retroalimentación constante con el cliente.

- **2.3.3. Comparación Ágil vs. Dirigido por Plan**:
  - [Comparación] **Dirigido por Plan**: **Etapas separadas, documentación formal, iteración dentro de actividades. Más adecuado para sistemas grandes y críticos con requisitos estables.**
  - [Comparación] **Ágil**: **Diseño e implementación como actividades centrales, iteración a través de actividades. Requisitos y diseño se desarrollan conjuntamente. Documentación just-in-time. Ideal para sistemas empresariales con requisitos volátiles.**
  - [Buena práctica] La mayoría de los proyectos reales combinan elementos de ambos enfoques, buscando lo mejor de cada uno. [Referencia].
- **2.3.4. Otros enfoques de proceso (CBSE, MDD, Métodos Formales)**:
  - [Definición] **Ingeniería de Software Basada en Componentes (CBSE)**: Enfocada en la reutilización de componentes de software ya existentes. Implica identificar, construir, catalogar y diseminar componentes. [Referencia].
  - [Definición] **Modelado Dirigido por Modelos (MDD/MDE)**: Creación de modelos a diferentes niveles de abstracción desde los cuales se puede generar el sistema ejecutable. [Referencia].
  - [Definición] **Métodos Formales**: Utilizan notación matemática rigurosa para especificar, desarrollar y verificar el software, asegurando la corrección del sistema. Muy adecuados para sistemas con altos requerimientos de seguridad o fiabilidad.
- **2.3.5. Estándares de Software y Mejora de Procesos**:
  - [Definición] Los **estándares de software** definen prácticas y convenciones. Ayudan a la continuidad y a la comprensión de nuevos miembros del equipo.
  - [Ejemplo] ISO 9001 es un marco para establecer estándares de calidad, no un estándar de desarrollo de software en sí.
  - [Riesgo] Pueden ser percibidos como burocráticos y poco relevantes por los ingenieros, especialmente si implican documentación manual y tediosa.
  - [Definición] La **Mejora de Procesos (MPS)** es un ciclo continuo de medir, analizar y cambiar los procesos para mejorar la calidad del producto y del proceso. Se basa en la idea de Deming de

control estadístico de calidad.

- [Métrica] El enfoque GQM (Goal-Question-Metric) ayuda a definir qué datos deben recopilarse y analizarse para la mejora de procesos.

2.4. El Proyecto (Project)

[Síntesis] **La gestión de proyectos de software es una actividad transversal que guía el proyecto desde antes de las actividades técnicas y durante todo el ciclo de vida.** Implica una planificación detallada, una administración proactiva de riesgos y una medición constante del progreso y la calidad. [Referencia].

- **2.4.1. Planificación del Proyecto**:
  - [Definición] Implica crear un "mapa" detallado del proyecto, definiendo tareas, riesgos, recursos y calendarios. Este plan se afina a medida que se conocen más detalles de los requisitos.
  - [Buena práctica] Principios de planificación: *entender el ámbito, dividir el trabajo, involucrar al equipo, iterar, ser flexible, gestionar el cambio, evaluar riesgos, y asegurar que los productos del trabajo añadan valor.*
  - [Conexiones] Las herramientas de calendarización del proyecto (como PERT y CPM) pueden aplicarse al desarrollo de software para definir tareas, dependencias, asignar recursos y monitorear.
- **2.4.2. Administración del Riesgo**:
  - [Definición] Actividad fundamental para **identificar, valorar, monitorear y mitigar los riesgos que pueden afectar un proyecto.**
  - [Identificación] Crear **listas de verificación de riesgos**, examinando el plan y el ámbito del software.
  - [Proyección] Evaluar la probabilidad de ocurrencia y el impacto de un riesgo.
  - [Mitigación] Desarrollar **planes de contingencia** (ej. sustituir componentes defectuosos o buscar más información ante cambios de requisitos) . La minimización del riesgo es clave. [Referencia].
- **2.4.3. Medición y Métricas de Proyecto**:
  - [Definición] La **medición de software** es el *proceso de derivar valores numéricos o perfiles para atributos de componentes, sistemas o procesos de software.*
  - [Definición] Las **métricas de software** son **características medibles**. Se dividen en métricas de control (para gestión de procesos) y de predicción (para características del software).
  - [Propósitos] **Caracterizar el esfuerzo, evaluar el progreso, predecir resultados, e identificar oportunidades de mejora.**
  - [Métrica] Las **líneas de referencia de métricas** son datos históricos de proyectos anteriores que sirven para comparar y evaluar mejoras. Deben ser precisas, consistentes, recopiladas de varios proyectos y relevantes. *"No se puede controlar lo que no se mide"*. [Referencia].

2.5. Artefactos y Productos del Trabajo

[Síntesis] **Los artefactos, o productos del trabajo, son los resultados tangibles de las actividades del proyecto. Estos pueden ser desde el código y la documentación hasta los modelos y planes.** Su propósito es añadir valor a las actividades subsiguientes y ser comprensibles para una audiencia diversa.

[Definición] Los **artefactos** o **productos del trabajo** son los **resultados tangibles** de las actividades del proyecto. [Referencia].

- [Ejemplo] Programas de software, archivos de configuración, documentación (del sistema y de usuario), modelos de requerimientos, modelos de diseño (ej. UML), código fuente, casos de prueba,

planes de proyecto, y manuales de usuario.

- [Buena práctica] Deben añadir valor a otras actividades del proceso y ser comprensibles para una audiencia potencialmente amplia (otros desarrolladores, mantenedores, usuarios).

2.6. Líneas Base y Administración de Configuración de Software (ACS)

[Síntesis] **Una línea base es un hito de desarrollo que formaliza un conjunto de ítems de configuración revisados y aprobados, sirviendo como punto de referencia para el futuro**.

La Administración de Configuración de Software (ACS) es el **proceso clave para gestionar y controlar estos productos del trabajo a lo largo de su ciclo de vida**, incluyendo sus cambios, para garantizar la integridad y trazabilidad del sistema. [Referencia].

- [Definición] Una **línea base** es un **conjunto de ítems de configuración que han sido construidos y revisados formalmente,** y sobre los que se ha llegado a un acuerdo, sirviendo como referencia de madurez y como base para futuros desarrollos.
- [Definición] La **Administración de Configuración de Software (ACS)** es el **conjunto de actividades** que ***identifican, controlan y dan seguimiento*** a los **productos del trabajo de software y sus cambios a lo largo del ciclo de vida del proyecto.**
- [Importancia] Es crucial para rastrear la documentación y asegurar que se entrega el sistema correcto, especialmente en sistemas críticos.
- [Repositorio] Un repositorio debe mantener los ítems de configuración de software (ICS) relacionados con todas las versiones del software y permitir su ensamblaje en configuraciones específicas. Incluye contenido de gestión de proyectos, documentos, modelos, construcción, V&V y empresarial.
- [Conexiones] La ACS se considera parte de la gestión de calidad.

---

06 Estimaciones de SW

La estimación en proyectos de software es un desafío constante, pero su precisión es vital para una planificación efectiva. Permite la **toma de decisiones informadas sobre el alcance**, **el presupuesto, el cronograma y la asignación de recursos.**

Importancia de la Estimación

[Síntesis] La estimación de software es **una de las actividades más desafiantes pero cruciales en la ingeniería de software**. Es fundamental para la planificación exitosa del proyecto, permitiendo a los gerentes tomar decisiones informadas sobre el alcance, presupuesto y cronograma. **Una buena estimación minimiza riesgos** y asegura una entrega de calidad. Mientras mas información tenga para estimar, menos incertidumbre y mayor certeza tendrá la misma. [Referencia].

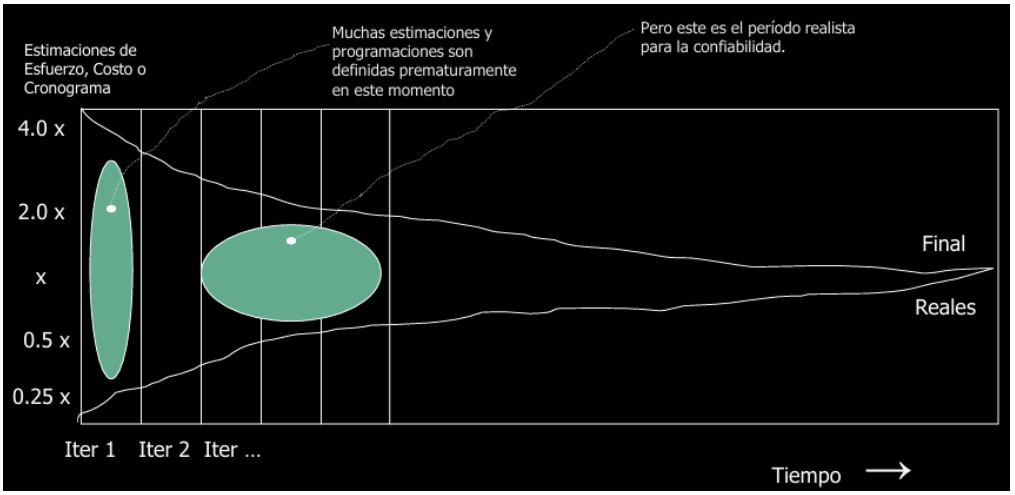
> 🚫 Las estimaciones NO SON compromisos. Por definición una estimación no es precisa.
>
> Estimar NO es planear.
>
> Las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
>
> A mayor diferencia entre lo estimado y lo planeado mayor riesgo.

- **Fundamentos**: La estimación es fundamental para:
  - **Planificación**: Permite crear planes de proyecto significativos que guíen al equipo.

- **Control**: Facilita el monitoreo del progreso del proyecto y la comparación de los avances y costos reales con lo planificado.
- **Calidad y Entrega**: Ayuda a producir sistemas de alta calidad y a entregarlos a tiempo.
- **Gestión del Riesgo**: Proporciona información necesaria para evaluar y mitigar riesgos .
- **Toma de Decisiones**: Apoya las decisiones sobre personal, presupuesto y recursos.

- [Buena práctica] Sin una estimación adecuada, "no se puede controlar lo que no se mide". La experiencia muestra que "cuanto más se conozca, mejor se estimará" [No encontrado en fuentes como cita explícita, pero inferido del contexto de la importancia del conocimiento].

Parámetros y Costos de Estimación.



[Síntesis] Los **costos de un proyecto de software se basan principalmente en el esfuerzo humano, que suele ser el rubro más grande**. Además, se deben considerar los **costos de hardware, software, viajes y capacitación**. Es esencial añadir una contingencia significativa a las estimaciones para absorber el optimismo inicial, los imprevistos, y contemplar el idealismo de la estimación, para comprender de que por definicion la misma no es precisa, como por ejemplo proponer 5 horas de trabajo y que al final sean 8 horas realmente porque no se contemplo el almuerzo, los recreos, falta de atencion, etc. [Referencia].

- **Parámetros Clave**: Los costos de un proyecto de software se calculan principalmente en base a tres parámetros:
  1. **Costos de esfuerzo**: Constituyen la mayor parte del presupuesto en la mayoría de los proyectos. Incluyen los salarios de ingenieros y administradores. El esfuerzo se suele medir en meses-persona.
  2. **Costos de hardware y software**: Abarcan la adquisición y mantenimiento de equipos, licencias de middleware y plataformas.
  3. **Costos de viajes y capacitación**: Aunque a menudo menores, el tiempo dedicado a viajar puede ser significativo y aumentar los costos de esfuerzo. Las herramientas de reunión electrónicas pueden ayudar a reducir estos costos.
- [Riesgo] Es fundamental estimar el esfuerzo total (en meses-hombre) y añadir una **contingencia significativa** (tiempo y esfuerzo adicionales) para cubrir posibles optimismos iniciales.

Técnicas de Estimación

[Síntesis] Las técnicas de estimación se dividen en dos categorías principales: **descomposición, que implica desglosar el software en partes y estimar cada una;**

y modelado empírico, que utiliza fórmulas matemáticas derivadas de datos históricos. También hay **enfoques basados en procesos.** [Referencia].

Mapa de métodos.

Las transparencias listan cinco familias de métodos:

(1) **basados en la experiencia**, dentro de "experiencia" están: datos históricos, juicio experto (puro / Wideband-Delphi) y analogía.

(2) **basados exclusivamente en los recursos.**

(3) **basados exclusivamente en el mercado.**

(4) **basados en componentes del producto o en el proceso.**

(5) **métodos algorítmicos.**

---

1) Basados en la experiencia

1.1 Datos históricos

Qué es: usar datos de proyectos anteriores (de la industria, de tu organización o del mismo proyecto en etapas pasadas) para estimar tamaño/tiempo/esfuerzo del actual. (Industry Data / Historical Data / Project Data).

Cómo hacerlo (paso a paso):

1. Reuní la línea base: qué métricas vas a mirar (p. ej., esfuerzo PM, duración, defectos).
2. Normalizá (mismo tipo de trabajo, tecnología y definiciones).
3. Buscá comparables *reales* (similar alcance y contexto).
4. Ajustá por diferencias visibles (dominio, equipo, herramientas).
5. Convertí tamaño→esfuerzo con la productividad histórica (p. ej., PF/PM o casos de uso/PM).

   Tip: este método transfiere el conocimiento del individuo a la organización.

> Ejemplo corto (propio): el año pasado "App A" costó 12 PM para 24 CU (casos de uso) ⇒ 2 CU/PM. Tu proyecto tiene 30 CU similares ⇒ ~15 PM (antes de ajustes).

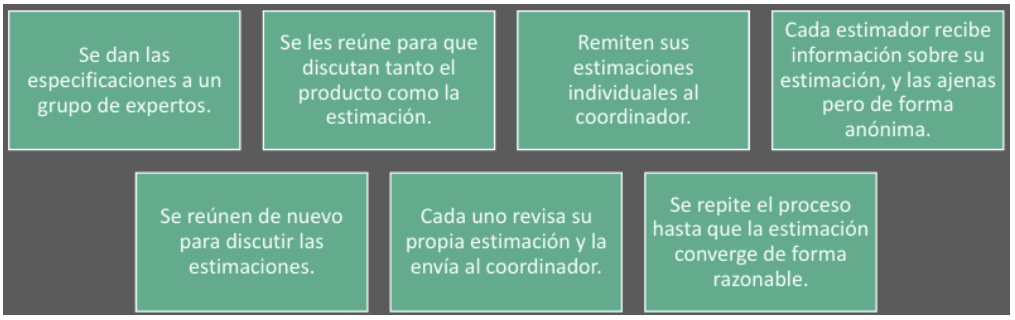
1.2 Juicio experto (puro)

Qué es: un experto lee las especificaciones y estima. Es lo más usado en la práctica. Problema: si el experto se va, perdés la capacidad de estimar.

Cómo estructurarlo bien: descomponé en tareas "estimables", aplicá PERT y usá checklist para cobertura. Fórmula PERT: **(o + 4·h + p) / 6** (optimista = o, habitual = h, pesimista = p).

> Ejemplo PERT (propio): o=6h, h=10h, p=18h ⇒ (6+40+18)/6 = 10,67h.

1.3 Juicio experto (Wideband-Delphi)



Qué es: como el juicio experto, pero **grupal** y anónimo por rondas; las estimaciones en grupo suelen ser mejores que las individuales. Es la base del Poker Estimation en Agile

Dinámica (paso a paso real):

1. Se envían especificaciones a varios expertos.
2. Discusión breve del alcance.
3. Cada uno envía su estimación al coordinador.
4. Se comparte el resumen (anónimo) y se debate.
5. Se re-estima.
6. Repetir hasta converger o aplicar PERT/desvío estándar si no converge.

1.4 Analogía

Qué es: comparás con *proyectos parecidos* (tamaño, complejidad, tecnología, equipo, nº de usuarios, etc.). Suele ser el método con **más error** si la analogía es pobre.

Cómo hacerlo:

1. Definí los factores de similitud (antes de buscar comparables).
2. Elegí 1–3 proyectos realmente cercanos.
3. Ajustá con coeficientes simples por cada diferencia relevante (p. ej., +15% si hay más usuarios, +10% por nueva tecnología). *(Ajustes: desarrollo propio.)*

---

2) Basados **exclusivamente** en los recursos

Qué es: estimar "**en función de cuánto personal tengo y por cuánto tiempo**". Ojo: el trabajo tiende a expandirse para consumir los recursos disponibles (Ley de Parkinson).

Cuándo usarlo: como "***cota realista de entrega***" cuando el presupuesto/plantel está fijado y no podés moverlo.

Riesgo típico: ignorar alcance y complejidad → fechas "forzadas".

---

3) Método **exclusivamente** de mercado (capacidad del cliente)

Qué es: se estima/propone en función de la **capacidad de pago** y lo que el cliente/mercado acepta, más que por el costo real. (A veces se cobra menos para ser elegidos).

Notas: en el resumen de cátedra se menciona como "basados en la capacidad del cliente".

Uso típico: ofertas comerciales, licitaciones, MVP con tope de presupuesto.

Riesgo: subestimar y comprometer calidad/scope.

---

4) Basados en **componentes del producto** o en el **proceso**

Qué es: estimar "por descomposición" (top-down/bottom-up) y/o por **actividades del proceso**.

Cómo hacerlo (paso a paso):

- **Bottom-up:**
  1. Hacé WBS hasta tareas de 4–16h (o "work packages").
  2. Estimá cada tarea (PERT/experto).
  3. Sumá y aplicá reservas (riesgo/contingencia). *(estructura: desarrollo propio; WBS y PERT ya vistos)*
- **Top-down:**
  1. Estimá por "bloques" (módulos/épicas).
  2. Repartí por regla de tres a subcomponentes.
  3. Refiná sucesivamente (rolling wave). *(desarrollo propio)*
- **Por proceso:** asigná esfuerzo por actividad (requisitos, diseño, construcción, pruebas, despliegue) para cada funcionalidad y sumás. *(enfoque citado por la cátedra como "en el proceso"; detalle operativo: desarrollo propio)*

---

5) Métodos **algorítmicos**

Qué es: usan fórmulas con parámetros cuantitativos (tamaño, complejidad, conocimiento de dominio). Devuelven esfuerzo y/o duración; aunque sean matemáticos, **no garantizan** exactitud.

Cómo se aplican (genérico):

1. Elegí la medida de tamaño (PF, CU, historias, etc.).
2. Valorá multiplicadores/ajustadores (p. ej., experiencia del equipo, restricciones, riesgos).
3. Corré el modelo y calibrá con datos históricos propios (factor de ajuste). *(pasos genéricos: desarrollo propio; el PDF enfatiza entradas cuantitativas y factores multiplicativos)*

Resumiendo las estimaciones

- **Cinco familias**: experiencia, recursos, mercado, componentes/proceso y algorítmicos.
- **Experiencia** incluye: históricos, juicio experto (PERT y checklist) y Delphi; **analogía** sólo si hay comparables sólidos.
- **Recursos/mercado**: útiles con presupuestos/planteles fijos, pero riesgosos si se ignora alcance.
- **Componentes/proceso**: descomponer y sumar (top-down / bottom-up) o repartir por actividades del proceso.
- **Algorítmicos**: usan entradas cuantitativas y multiplicadores; requieren **calibración** con datos propios.

> Nota de transparencia: cuando detallo "pasos operativos" (p. ej., reservas, rolling-wave, coeficientes de ajuste), es desarrollo propio para que puedas aplicarlos en práctica; lo central y exigible en el examen sigue la estructura y definiciones de las transparencias y resúmenes de la cátedra citados arriba.

Secciones Transversales

Glosario Crítico

- **ACS (Administración de Configuración de Software)**: Conjunto de actividades para identificar, controlar y seguir los productos de trabajo de software y sus cambios a lo largo del ciclo de vida.
- **ACS (Aseguramiento de la Calidad del Software)**: Definición de procesos y estándares para obtener productos de alta calidad y la verificación de su aplicación.
- **Ágil (Métodos Ágiles)**: Enfoques de desarrollo de software que valoran individuos, software operativo, colaboración con el cliente y respuesta al cambio.
- **Ámbito del Software**: El alcance del producto, incluyendo funciones, características, datos, rendimiento y restricciones.
- **Artefactos / Productos del Trabajo**: Resultados tangibles de las actividades del proyecto (ej. código, documentación, modelos).
- **Caso de Uso**: Descripción de un escenario específico de interacción del usuario con el sistema.
- **COCOMO**: Modelo de estimación de costos algorítmico conocido.
- **Contingencia**: Tiempo y esfuerzo adicionales añadidos a la estimación para cubrir posibles imprevistos.
- **Ingeniería de Requerimientos**: Proceso de comprender y definir los servicios y restricciones de un sistema.
- **Juego de Planeación**: Método ágil de estimación de XP que involucra a todo el equipo y al cliente para priorizar y estimar historias de usuario.
- **Línea Base**: Especificación o producto formalmente revisado y acordado que sirve como referencia para desarrollos futuros.
- **LOC (Líneas de Código)**: Métrica de tamaño de software basada en el recuento de líneas de código fuente.
- **Métricas de Software**: Características medibles de un sistema, documentación o proceso de desarrollo.
- **PF (Puntos de Función)**: Métrica de tamaño de software que mide la funcionalidad desde la perspectiva del usuario .
- **Proceso de Software**: Colección de actividades, acciones y tareas para crear un producto de software.
- **Proyecto de Software**: Esfuerzo temporal para crear un producto, servicio o resultado único.
- **Requerimientos Funcionales**: Describen los servicios que el sistema debe proporcionar.
- **Requerimientos No Funcionales**: Especifican restricciones operacionales y de desarrollo (ej. fiabilidad, rendimiento, seguridad).
- **Riesgo**: Algo que podría salir mal en un proyecto, con probabilidad e impacto asociado.
- **SRS (Documento de Requerimientos de Software)**: Comunicado oficial de lo que deben implementar los desarrolladores.
- **Velocidad (XP)**: Número de puntos de esfuerzo implementados por el equipo por día, usado para ajustar las estimaciones.

Cuadros Comparativos

Característica	Enfoque Tradicional (Ej. Cascada/Dirigido por Plan)	Enfoque Ágil (Ej. XP/Scrum)
Documentación	Exhaustiva y formal, generada en etapas tempranas. Contrato formal.	Ligera o "just-in-time", se valora el software funcionando sobre la documentación. Historias de usuario.
Respuesta al Cambio	Inflexible; los cambios tardíos son costosos y difíciles de integrar.	Adaptable y bienvenida; el cambio es la norma.
Iteración	Dentro de las fases (ej. iteración en diseño); fases secuenciales con revisiones al final.	A través de actividades (ej. requisitos y diseño en conjunto); iteraciones cortas y rápidas.
Participación del Cliente	Formal, en hitos clave (ej. aprobación de requisitos).	Colaboración continua y cercana durante todo el ciclo.
Riesgo	Gestionado explícitamente, pero puede no ser tan adaptativo a nuevos riesgos.	Reconocido e implícitamente gestionado a través de la adaptación constante.
Equipos	Jerárquicos, roles definidos, menos autonomía.	Pequeños, autoorganizados, alta motivación, roles fluidos.
Estimación	Basada en tamaños (LOC, PF), modelado algorítmico, detallada al inicio.	Basada en "historias de usuario", "puntos de esfuerzo" y "velocidad", refinamiento continuo.
Adecuado para	Sistemas grandes, críticos, con requisitos estables y alta necesidad de formalidad (ej. aeroespacial).	Sistemas empresariales, requisitos volátiles, equipos pequeños y estables (ej. startups, desarrollo web).

Checklist de Buenas Prácticas

- **Personal**:
  - Crear un equipo con un equilibrio de habilidades técnicas y personalidades complementarias.
  - Fomentar la comunicación abierta y la colaboración constante.
  - Proporcionar capacitación continua al personal.
  - Liderar con el ejemplo y empoderar al equipo.
- **Producto**:
  - Definir el ámbito del software de forma clara y cuantitativa antes de la estimación.
  - Utilizar especificaciones de requerimientos (SRS) claras, no ambiguas y completas.
  - Gestionar activamente los cambios en los requerimientos.
  - Planificar la calidad desde el inicio del proyecto y asegurarla a lo largo de todas las fases.
- **Proceso**:
  - Adaptar el modelo de proceso a las necesidades específicas del proyecto, el equipo y el producto.
  - Implementar actividades sombrilla (ACS, revisiones, medición, gestión de riesgos) de forma continua.
  - Realizar revisiones técnicas formales para detectar errores tempranamente.
  - Promover una cultura de calidad en todo el equipo de desarrollo.
- **Proyecto**:
  - Establecer objetivos claros y expectativas realistas.
  - Dividir el trabajo en tareas manejables y asignarlas de manera eficiente.
  - Identificar, evaluar y mitigar los riesgos de forma proactiva.
  - Utilizar métricas para monitorear el progreso, la productividad y la calidad.

- Establecer líneas base para formalizar hitos y controlar cambios.

Triple restricción en Ágil

1) ¿Qué es la triple restricción?

La “triple restricción” (o triángulo de hierro) equilibra **alcance**, **tiempo** y **costo**; el balance de estos tres factores impacta directamente en la **calidad** del proyecto. El rol de la jefatura de proyecto es mantener ese equilibrio para cumplir objetivos en el tiempo y presupuesto definidos.

2) ¿Qué cambia en Ágil?

En el enfoque **tradicional** se suele **fijar el alcance** y, a partir de allí, derivar tiempo y costo.

En **Ágil** se **fijan tiempo (iteraciones/sprints) y recursos (equipo/capacidad)**, y el **alcance** se **negocia** y **prioriza** en cada iteración para maximizar valor de negocio. Esto permite ajustar qué se entrega al inicio de cada sprint según la prioridad del Product Owner.

> Regla práctica ágil: Tiempo fijo + Equipo fijo → Alcance variable por iteración (priorizado por valor).

3) El “triángulo ágil” (lectura recomendada)

Tu material introduce el **triángulo ágil** como contrapunto al clásico: se centra en **entregar valor** temprano y frecuente, con decisiones “just in time” y priorización continua por parte del PO.

4) Implicancias prácticas (para tu resumen)

- **Planificación**: se planifica a capacidad del equipo y duración fija; el backlog “entra” según prioridad y tamaño.
- **Gestión del alcance**: el **PO** puede **reordenar** requerimientos entre iteraciones sin romper el plan de fechas/recursos.
- **Calidad**: no es “la cuarta punta”, sino la consecuencia del equilibrio y de entregar **software funcionando** en cada iteración.
- **Valor de negocio**: el objetivo no es “más funcionalidades” sino **más valor**; la funcionalidad es el **medio** para entregar ese valor.

5) Cómo explicarlo en un examen (formulita)

> En Ágil, la triple restricción se gestiona fijando tiempo y recursos (sprints y equipo) y ajustando alcance por prioridad de negocio en cada iteración; así se maximiza valor sin mover fechas ni equipo, manteniendo el equilibrio que determina la calidad.

Mini-ejemplo

- Sprint de 2 semanas, equipo de 6 personas (**fijo**).
- Backlog priorizado por PO: entran las historias que **entran en la capacidad**; si aparece algo más valioso, el PO **reordena** para el próximo sprint (**alcance variable**).

En una línea (para tu apunte)

Ágil = Tiempo fijo + Equipo fijo + Alcance variable (priorizado por valor) ⇒ Calidad como resultado del equilibrio.